ALGORITMOS E ESTRUTURAS DE DADOS I

ATIVIDADE PRÁTICA

Listas Duplamente Encadeadas em C

Discente(s): _____

Turma: _____ **Data:** _____

1 Contextualização

Uma lista duplamente encadeada é uma estrutura de dados dinâmica na qual cada elemento, denominado **nó**, armazena um dado e dois ponteiros:

- `anterior`: aponta para o nó anterior;
- `proximo`: aponta para o próximo nó.

Diferentemente de uma lista simplesmente encadeada, essa estrutura permite percorrer os elementos em duas direções: do início para o final e do final para o início.

2 Atividade

Desenvolva um programa em C que implemente um sistema de gerenciamento de uma lista de números inteiros utilizando uma estrutura duplamente encadeada.

O programa deverá permitir que o usuário insira, consulte, remova e visualize os elementos armazenados.

A lista deverá permitir percorrimento tanto do início para o final quanto do final para o início.

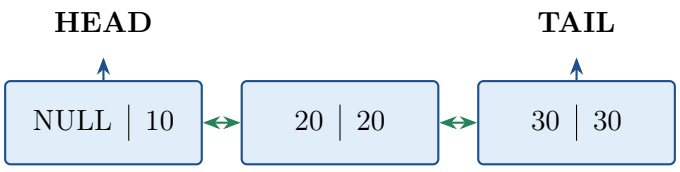
A estrutura deverá possuir três campos:

```c
typedef struct No {
    int dado;
    struct No *anterior;
    struct No *proximo;
} No;
```

Cada nó contém um valor inteiro e dois ponteiros responsáveis pela ligação com os nós vizinhos.

2.1 Representação gráfica

Considere uma lista com os valores 10, 20 e 30.



Representação conceitual dos nós

Observação: cada nó possui um campo de dado, um ponteiro anterior e um ponteiro próximo. As ligações entre os nós são bidirecionais.

A representação lógica da lista é:

$$NULL \leftrightarrow 10 \leftrightarrow 20 \leftrightarrow 30 \leftrightarrow NULL$$

3 Requisitos da implementação

O programa deverá implementar obrigatoriamente as operações descritas a seguir.

3.1 1. Inicialização da lista

Criar uma lista vazia, inicializando os ponteiros necessários.

3.2 2. Inserção no início

Implementar uma função que receba um valor inteiro e insira um novo nó no início da lista.

A operação deverá atualizar corretamente:

- O ponteiro para o primeiro elemento;
- O ponteiro `anterior` do antigo primeiro nó;
- Os ponteiros do novo nó.

3.3 3. Inserção no final

Implementar uma função que insira um novo elemento no final da lista.

A operação deverá atualizar corretamente os ponteiros do último nó e do novo elemento.

3.4 4. Inserção em uma posição específica

Implementar uma função que permita inserir um elemento em uma posição informada pelo usuário.

Considere que a primeira posição da lista é 1.

O programa deverá tratar posições inválidas.

3.5 5. Remoção do início

Implementar uma função que remova o primeiro elemento da lista.

Deverá tratar o caso em que a lista está vazia e o caso em que existe apenas um elemento.

3.6 6. Remoção do final

Implementar uma função que remova o último elemento da lista.

Deverá atualizar os ponteiros e liberar a memória ocupada pelo nó removido.

3.7 7. Remoção de um valor

Implementar uma função que procure um determinado valor e remova sua primeira ocorrência na lista.

Caso o valor não exista, o programa deverá informar ao usuário.

3.8 8. Busca de elementos

Implementar uma função que procure um valor na lista e informe se ele foi encontrado.

3.9 9. Percorrimento da lista

Implementar duas funções:

- Percorrer e imprimir a lista do início para o final;
- Percorrer e imprimir a lista do final para o início.

3.10 10. Contagem de elementos

Implementar uma função que retorne a quantidade de nós existentes na lista.

3.11 11. Liberação da memória

Implementar uma função que percorra a lista, libere todos os nós alocados dinamicamente e deixe a lista vazia.

4 Estrutura inicial do código

O grupo deverá utilizar a estrutura apresentada a seguir como ponto de partida.

As funções deverão ser implementadas pelos próprios estudantes.

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *anterior;
    struct No *proximo;
} No;

typedef struct Lista {
    No *inicio;
    No *fim;
} Lista;

// Inicializar lista
void inicializar(Lista *lista);

// Inserir elemento no inicio
void inserirInicio(Lista *lista, int valor);

// Inserir elemento no final
void inserirFinal(Lista *lista, int valor);

// Inserir em uma posicao
void inserirPosicao(Lista *lista,
                    int valor, int posicao);

// Remover do inicio
void removerInicio(Lista *lista);

// Remover do final
void removerFinal(Lista *lista);

// Remover um valor
void removerValor(Lista *lista, int valor);

// Buscar um valor
No *buscar(Lista *lista, int valor);

// Imprimir do inicio ao final
void imprimirDireto(Lista *lista);

// Imprimir do final ao inicio
void imprimirInverso(Lista *lista);

```

```c
// Contar elementos
int contarElementos(Lista *lista);

// Liberar memoria
void liberarLista(Lista *lista);

int main() {

    Lista lista;

    inicializar(&lista);

    // Implementar menu e chamadas das funcoes

    liberarLista(&lista);

    return 0;
}
```

5 Regras e cuidados na implementação

Durante o desenvolvimento, os grupos deverão observar as seguintes regras:

1. Utilizar obrigatoriamente a linguagem C;
2. Utilizar estruturas (`struct`);
3. Utilizar ponteiros para conectar os nós;
4. Utilizar `malloc()` para alocação dinâmica;
5. Utilizar `free()` para liberar a memória;
6. Implementar cada operação em uma função;
7. Não utilizar vetores para armazenar os elementos da lista;
8. Não utilizar bibliotecas prontas de listas;
9. Verificar os casos de lista vazia e de um único nó;
10. Manter a consistência dos ponteiros após cada operação.

6 Testes obrigatórios

O programa deverá ser testado considerando pelo menos as seguintes situações:

Teste	Situação	Resultado esperado
1	Inicializar lista	Lista vazia
2	Inserir no início	Novo elemento como primeiro nó
3	Inserir no final	Novo elemento como último nó
4	Inserir em posição válida	Elemento na posição solicitada
5	Inserir em posição inválida	Operação rejeitada
6	Remover do início	Primeiro nó removido
7	Remover do final	Último nó removido
8	Remover valor existente	Valor removido
9	Remover valor inexistente	Mensagem informativa
10	Buscar elemento	Resultado correto da busca
11	Percorrer em ambas as direções	Sequências inversas corretas
12	Remover todos os elementos	Lista vazia, sem vazamento de memória

7 Seminário: apresentação e defesa do código

Cada grupo deverá apresentar o programa desenvolvido em um seminário.

A apresentação deverá demonstrar que os integrantes compreendem o funcionamento da estrutura e conseguem justificar as decisões de implementação.

7.1 Parte 1 – Fundamentação teórica

O grupo deverá explicar:

1. O que é uma lista duplamente encadeada?
2. Qual a diferença entre lista simplesmente e duplamente encadeada?
3. Qual a função dos ponteiros `anterior` e `proximo`?
4. Qual a função dos ponteiros `inicio` e `fim`?
5. Quais são as vantagens e desvantagens dessa estrutura?

7.2 Parte 2 – Explicação da implementação

O grupo deverá apresentar o código e explicar:

1. Como um novo nó é criado?
2. Como os nós são conectados?
3. Como ocorre uma inserção no início?
4. Como ocorre uma inserção no final?
5. Como os ponteiros são atualizados durante uma remoção?
6. Como funciona o percorrimento inverso?
7. Como a memória é liberada?

7.3 Parte 3 – Demonstração prática

O grupo deverá executar o programa e demonstrar as seguintes operações:

1. Criar uma lista;
2. Inserir pelo menos quatro elementos;
3. Mostrar a lista nos dois sentidos;
4. Inserir um elemento em uma posição específica;
5. Remover um elemento;
6. Buscar um valor;
7. Remover todos os elementos;
8. Demonstrar o comportamento da lista vazia.

7.4 Parte 4 – Defesa do código

Durante o seminário, o professor poderá selecionar qualquer integrante para responder perguntas sobre a implementação.

As perguntas poderão envolver:

- Justificativa da utilização de ponteiros duplos ou de uma estrutura que armazena início e fim;
- Atualização dos ponteiros em diferentes operações;
- Tratamento de situações especiais;
- Análise da complexidade;
- Identificação de possíveis erros no código;
- Consequências de não utilizar `free()`.

Atenção: defesa individual

Todos os integrantes deverão conhecer a implementação completa.
Não será suficiente explicar apenas a função desenvolvida individualmente.
O professor poderá solicitar que qualquer integrante explique, localize ou modifique uma parte do código durante a apresentação.

8 Orientações para o seminário

- Todos os integrantes deverão participar da apresentação;
- O grupo deverá explicar o código desenvolvido, e não apenas apresentar slides;
- A demonstração deverá utilizar o programa efetivamente implementado;
- Os integrantes deverão estar preparados para responder perguntas sobre qualquer função;
- Alterações simples no código poderão ser solicitadas durante a defesa.

9 Reflexão final

Após concluir a atividade, responda:

1. Em quais situações uma lista duplamente encadeada pode ser mais interessante que uma lista simplesmente encadeada?
2. Quais são os custos adicionais de memória associados à utilização de dois ponteiros por nó?
3. Como a utilização dos ponteiros `inicio` e `fim` pode melhorar o desempenho de determinadas operações?
4. Qual foi a maior dificuldade encontrada durante a implementação e como ela foi resolvida?